

Performance of Heuristics for Minimum-Weight Perfect Matching on Surface Codes

Gene Yang

May 2026

Abstract

Surface codes are a promising way to perform quantum error correction. Compared to other error-correction schemes, surface codes have a relatively high threshold, allowing for arbitrarily low logical error rates through an increase in the number of qubits. Error correction in the surface code is done classically: given the syndrome generated by measurements, recovery chains are found and applied. This step depends on specialized algorithms, such as Edmond's blossom algorithm in Minimum-Weight Perfect Matching (MWPM). In this paper, three heuristics for MWPM are introduced: greedy, nearest-neighbor, and space-filling curve. The effectiveness of these heuristics in the case of a $L \times L$ surface code is analyzed. In a simulation of independent and correlated errors, the greedy and nearest-neighbor heuristics outperform the blossom algorithm while keeping the cost to within 5% and 8% of the optimal. Though the simulation here is crude, it shows the potential of using heuristics to speed up MWPM instead of finding an exact solution.

Contents

1	Surface Codes	2
1.1	Introduction	2
1.2	Thresholds	2
1.3	Stabilizer Formalism	3
1.3.1	Toric Code	3
1.3.2	Planar Code	4
1.4	Error Correction	4
2	Decoding Algorithms	5
2.1	Minimum-Weight Perfect Matching	5
2.2	Union-Find	6
2.3	Neural Networks	6
3	Heuristics for Minimum-Weight Perfect Matching	6
3.1	Greedy Heuristic	6
3.2	Nearest-Neighbor Subgraphs	6
3.3	Space-Filling Curves	7
3.4	Other Heuristics	7
4	Analysis of Heuristics Performance	8
4.1	Independent Error Model	8
4.2	Correlated Errors Model	10
5	Conclusion	11

1 Surface Codes

1.1 Introduction

The surface code is a type of quantum error correcting code first described by Kitaev [1]. In the basic form of surface codes, qubits are encoded on the edges of a $L \times L$ lattice with edges identified. This results in a toric geometry, on which errors can occur and form longer connected chains. However, certain simultaneous measurements of neighboring qubits can pinpoint the location of these errors, specifically the endpoints of error chains. Using classical algorithms, a recovery chain that can correct the error chain is found and applied. Speed is critical for the classical algorithm: errors must be corrected fast enough to keep up with the quantum device [2].

The main benefit of surface codes is its topological nature: as long as the recovery chain and error chain are homologically equivalent, it doesn't matter what recovery chain is used [3]. This makes the surface code extremely tolerant to local errors. However, due to the lattice structure, surface codes require a large number of physical qubits to encode even one logical qubit. At minimum, 13 physical qubits can form a single logical qubit, but to achieve fault-tolerant computation, an order of 10^3 to 10^4 physical qubits are estimated to be required for each logical qubit [4].

Since its formulation, several interesting advances have been made with regard to surface codes. It was shown that the toric geometry was unnecessary, and a planar surface with boundaries could be used instead [5]. In the presence of biased noise, Ataides et al. showed a significant improvement in error correction thresholds using a XZZX code, a variant of the surface code [6]. Recently, in 2023, Google announced that they had achieved a reduction in logical error rates by adding more physical qubits, marking a milestone in fault-tolerant quantum computation [7].

On the classical side of things, researchers have developed faster algorithms for finding recovery chains. Two candidates for such algorithms are Minimum-Weight Perfect Matching (MWPM) and Union-Find.

1.2 Thresholds

Quantum error correction is possible thanks to the threshold theorem, which roughly says that for error rates below the threshold, logical errors can be suppressed to an arbitrary degree [8]. Error rates smaller than the threshold require an increasingly small amount of physical qubits per logical qubit [4]. Therefore, the higher the threshold, the easier the code is to implement.

Surface codes tend to have larger thresholds than their counterparts, making them a good candidate for future quantum computers. The threshold depends on several factors, such as the physical implementation and decoding algorithm.

Code Type	Estimated Threshold
Steane code	$2 \times 10^{-3}\%$ [4]
Surface code, unbiased noise	1% [4]
Surface code, pure Z noise	10.9% [8, 9]
Surface code, pure Z noise with tailored decoder	43.7% [9]

1.3 Stabilizer Formalism

1.3.1 Toric Code

The toric code is the simplest form of the surface code. Qubits lie on a $L \times L$ lattice with opposite edges identified. This forms a toroidal geometry.

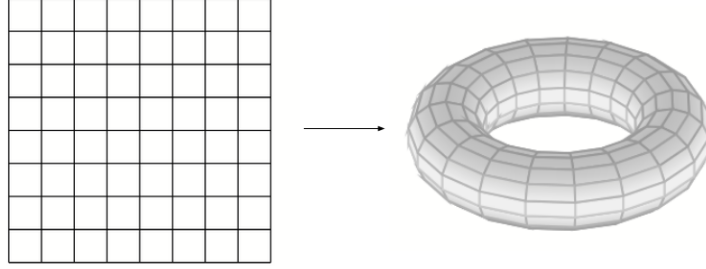


Figure 1: Square lattice with opposite edges identified.

Each edge can also be associated with two operators: X and Z . Since Y is a combination of X and Z , it isn't considered. Let v be a vertex and f be a face on the lattice, with ∂ as the boundary operator. The stabilizers of the toric code are defined as all possible combinations of the face and vertex operators [3]:

$$X_v = \prod_{e|v \in \partial e} X_e, \quad Z_f = \prod_{e \in \partial f} Z_e$$

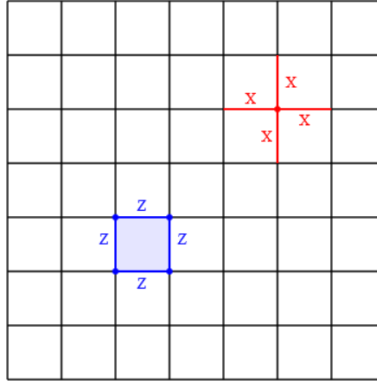


Figure 2: Stabilizers of the toric code.

Since they can only overlap on 0 or 2 qubits, the X and Z stabilizers commute. X and Z errors can be corrected in similar ways using stabilizers. For now, focus on the X errors.

Let $\mathcal{P}_n$ be a tensor product of Pauli matrices $\{X, Y, Z\}^{\otimes n}$. The stabilizer group is a subgroup $\mathcal{S} \in \mathcal{P}_n$ [10]. The codespace is the $+1$ eigenstate of $\mathcal{S}$, such that $S|\psi\rangle = |\psi\rangle$ for all $S \in \mathcal{S}$.

Contractible loops, those formed by combining stabilizers, leave the codespace unchanged. This is because every time we grow a contractible loop, the added stabilizer must intersect the contractible loop twice. Though it's easiest to see for Z stabilizers here, the X stabilizers can also form contractible loops in the dual lattice.

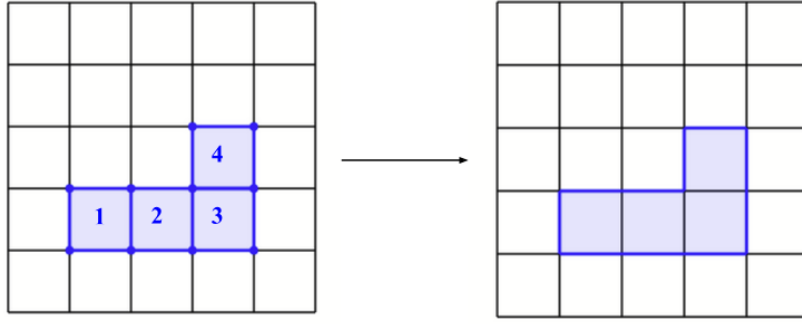


Figure 3: Intersecting stabilizers form a contractible loop in the lattice.

1.3.2 Planar Code

The toric geometry is difficult to implement physically. Instead, we can use a planar code, a sheet with rough and smooth boundaries. Details are the same as the toric code, except that stabilizers on the boundary depend on 3 measurements, not 4.

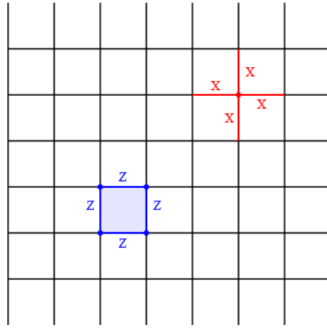


Figure 4: Stabilizers of the planar code.

Decoding algorithms for the toric code can easily be extended to the planar code. They must account for boundaries and the possibility of “isolated” syndromes.

1.4 Error Correction

Given some error chain $E \in \mathcal{P}_n$ of the surface code, we measure all X stabilizers and mark locations that are not in the codespace (the “syndrome”). Within a single error chain, the only locations of the syndrome are at the endpoints of the chain. In all other locations on the chain, each X stabilizer overlaps with two error qubits, resulting in an overall $(-1)^2 = 1$ phase that cancels out.

Given the set of syndromes due to an error chain E , our goal is to find a recovery path R between them such that $RE \in \mathcal{S}$. That is, if R and E form a contractible loop, it will leave the codespace unchanged and thus “undo” the error.

There’s also a chance that R and E form a non-contractible loop. Stabilizers can’t detect this, since each stabilizer overlaps with two error qubits. However, the state is still changed. This is an undetectable logical error, and we should avoid it in our decoding process. Mathematically, the non-contractible loops correspond to the first homology groups of a torus [3].

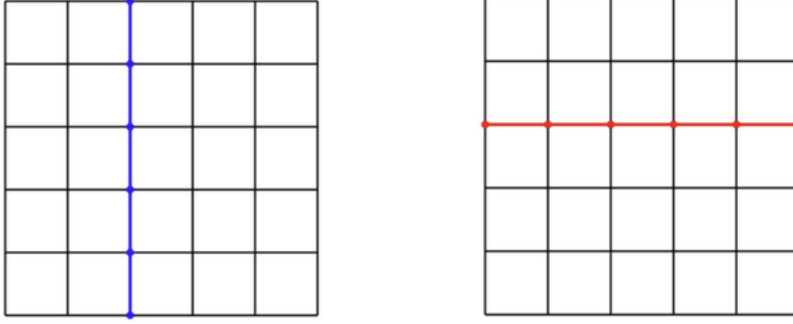


Figure 5: Non-contractible loops generated by Z stabilizers. X stabilizers can generate similar loops in the dual lattice.

The basic algorithm behind decoding is minimum-weight matching. For the set of all syndromes $\sigma(E)$ for a given error E , we seek to pair them in a way that minimizes the cumulative distance between each pair. A basic implementation might use Manhattan distance, for example, adapted for a torus. However, the distance can be calculated differently if the error rate is not uniform across qubits.

If the probability of the i -th qubit encountering an X error is p_i , then we can assign a weight $w_i = \log((1 - p_i)/p_i)$ to each edge in the lattice. Let $e_i = 0$ if the i -th qubit has no error, and $e_i = 1$ otherwise. The probability of the error occurring is multiplied over all edges [10]:

$$p(E) = \prod_i (1 - p_i)^{1-e_i} p_i^{e_i} = \prod_i (1 - p_i) \prod \left(\frac{p_i}{1 - p_i} \right)^{e_i}$$

The error E is chosen so that it corresponds to the syndrome and has highest probability $p(E)$.

2 Decoding Algorithms

2.1 Minimum-Weight Perfect Matching

Minimum-Weight Perfect Matching (MWPM) is a popular option for decoding surface codes. Given a graph of vertices and edges, MWPM can find a pairing between all vertices that minimizes the total “cost” (sum of distance between two paired nodes).

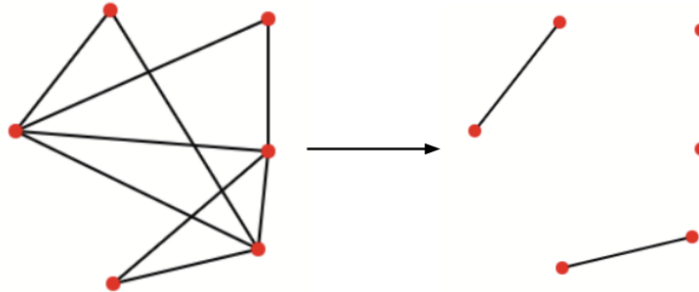


Figure 6: Example of MWPM. Edge weights not written.

MWPM can be implemented with Edmond’s blossom algorithm. Efficient implementations of the blossom algorithm have a worst-case time complexity of $O(N^3)$, where N is the number of vertices in the graph

[11]. This can be sped up significantly using parallelization or algorithm variants like the Sparse Blossom, introduced by Higgott in the Python PyMatching package [12].

2.2 Union-Find

The Union-Find algorithm is another candidate for decoding surface codes. “Clusters” are grown from each error and merged when they intersect. By using standard optimizations, like weighted union and path compression, the algorithm can be made to run in $O(N\alpha(N))$ time, where $\alpha(N)$ is the inverse Ackermann function [2]. For all practical purposes, this is linear.

Thus, the Union-Find algorithm seems to outperform MWPM. However, the Union-Find algorithm only finds an approximate solution for the recovery chain, unlike MWPM, which always finds the optimal solution [13].

2.3 Neural Networks

Other decoders have been proposed for surface codes. A recent suggestion is to use machine learning. By training networks on large datasets obtained by simulating different error rates, we can produce fairly accurate recovery chains [14]. A neural network decoder has a logical failure rate close to that of MWPM, but has the benefit of inherently accounting for different error rates [14].

3 Heuristics for Minimum-Weight Perfect Matching

Instead of finding ways to optimize MWPM, we can look at ways to speed up MWPM while sacrificing some accuracy. This modified MWPM might not find the optimal pairings, but it might run faster.

The three main heuristics introduced here were chosen for 1) ease of implementation, 2) simplicity, and 3) variety. The three heuristics cover the greedy, subgraph, and geometric strategies. Many other complicated heuristics can be constructed using a combination of these ideas.

3.1 Greedy Heuristic

One obvious heuristic is the greedy heuristic: always pairing the two closest edges. For N vertices, this runs in worst-case $O(N^2 \log N)$ after sorting the N^2 edges, an improvement over the $O(N^3)$ running time of the exact algorithm [15]. The ratio of the cost of matching produced by this heuristic to the optimal cost of matching is bounded by $O(\frac{4}{3}N^{\log(3/2)})$ [15]. Though this isn’t ideal, it may still be a good tradeoff especially for smaller values of N . To mitigate this, we could test all 4 orientations of the serpentine curve on the plane, then use the solution that gives minimum weight.

The greedy heuristic is one of the simplest to implement, requiring only a sorting combined with a check for unpaired vertices.

3.2 Nearest-Neighbor Subgraphs

The nearest-neighbor heuristic splits the total graph $G = (V, E)$ of syndromes into several subgraphs of nearest-neighbors. This is a subroutine of the more complicated Onethird and (t, k) heuristics [16, 17]. Each syndrome vertex is connected to its nearest neighbor, resulting in several subgraphs $G_i = (V_i, E_i)$ that can be solved independently with Edmond’s blossom algorithm. The blossom algorithm is applied again at the end for any unpaired vertices.

The idea behind the nearest-neighbor heuristic is to reduce the large $O(N^3)$ factor of blossom's algorithm by only applying it to subgraphs of small N . Nearest neighbors can be found in $O(N^2)$, and assuming k connected nearest-neighbor components, the heuristic has a rough upper bound of $O(N^2 + k(N/k)^3 + k^3)$. This tends to outperform the $O(N^3)$ time complexity of the blossom algorithm; however, the actual comparison will depend on the constant factors.

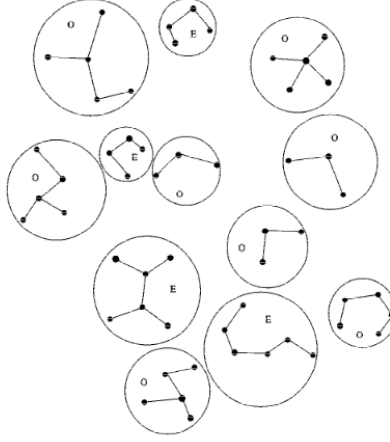


Figure 7: An example nearest-neighbor graph [17]

3.3 Space-Filling Curves

We can also exploit the planar nature of the surface code to create a geometric heuristic. Each cell (x, y) on the lattice can be parametrized according to its position on a spacefilling curve [18]. By sorting syndromes according to their position along this curve, neighboring syndromes can be paired reasonably accurately in $O(N \log N)$ time. Though this is faster than the previous heuristics, it comes at the cost of a large error ratio. In the case of Iri et al.'s serpentine curve, if syndrome vertices constantly appear the “bends” of the curve, the error ratio will be close to $O(N)$. This error bound can be reduced to $O(N/4)$ by minimizing over all 4 possible serpentine curves, at the cost of a fourfold increase in running time.

Options for the spacefilling curve range from the simple serpentine cell order of Iri et al. to the complex Hilbert curve introduced by Bartholdi and Platzman [19, 18]. For simplicity, the serpentine cell order of Iri et al. is used in the later simulation.

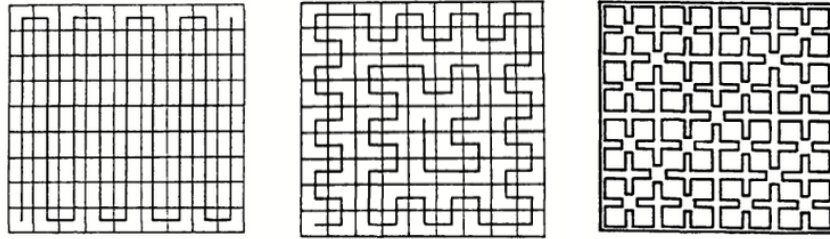


Figure 8: (Left to right) Serpentine cell order, spiral cell order, Hilbert curve [18]

3.4 Other Heuristics

Several other complex heuristics have been developed for the blossom algorithm. The Onethird heuristic, for example, splits a graph of vertices into several subgraphs that are solved iteratively. Let A be an exact

MWPM algorithm that runs in time $t_A(N)$ with error bound $f_A(N)$. For a graph with N vertices, for each $k \leq \log_3 N$, the Onethird heuristic can find pairings in time $O(\max\{N^2, t_A(3^{-k}N)\})$ with error bound $O((\frac{7}{3})^k (1 + f_A(3^{-k}N)) - 1)$ [16]. In the case of Edmond's blossom algorithm with $t_A(N) = N^3$ and $f_A(N) = 0$, the Onethird heuristic can run in $O(N^3/3^{3k})$ time with error bound $O((\frac{7}{3})^k - 1)$. The Onethird heuristic incorporates uses the nearest-neighbor heuristic given above.

Another heuristic is the (t, k) heuristic, a generalization of the greedy and Onethird heuristics [17]. Like the Onethird heuristic, the (t, k) heuristic works by splitting the graph into several subgraphs that are solved before being joined together. Other exact perfect matching algorithms can be used in the subprocesses of the (t, k) heuristic. In its form introduced in the paper, the (t, k) heuristic can obtain a solution with error ratio bound by $O(3(\log_3 \log_3 N)^{0.125} - 1)$ in $O(N^2)$ time. The (t, k) heuristic requires that edges in the graph fulfill the triangle inequality [17]. For a surface code on a lattice, this is satisfied.

Given the relatively small lattice sizes used for surface codes, these heuristics aren't considered in this paper.

4 Analysis of Heuristics Performance

A basic simulation of heuristic-based MWPM is done on a $L \times L$ lattice. Errors are distributed randomly on the lattice with probability p_e and the syndrome is found. From this syndrome, we can construct a dense graph of syndrome vertices and use different MWPM heuristics to find a reasonably optimal matching.

For simplicity, a simplified planar code model is used: edges aren't identified with each other. Additionally, we only consider cases with qubit errors that don't intersect the boundary. This ensures that every vertex in the syndrome is the endpoint of a loop, removing the edge cases of the algorithm. A complete algorithm that handles errors on the boundary could be achieved with a $O(N)$ check to remove errors sufficiently close to the boundary.

To avoid time differences due to data structure initialization, each heuristic is given an unsorted list of vertices and edges in the overall graph. The $O(N^3)$ implementation of Edmond's blossom algorithm was achieved using Python's NetworkX package [20]. For each lattice size L , the heuristics are ran 100 times and the results are averaged.

The code can be found at <https://github.com/gyang0/MWPM-Heuristics-Test>

4.1 Independent Error Model

We analyze a basic error model where X errors are applied randomly on the lattice with probability p . The 3 main heuristics (greedy, nearest-neighbor, space-filling) are compared with a pure blossom algorithm approach. The graphs for $p = 0.01$ and $p = 0.05$ are shown below in Figure 9 and Figure 10, respectively.

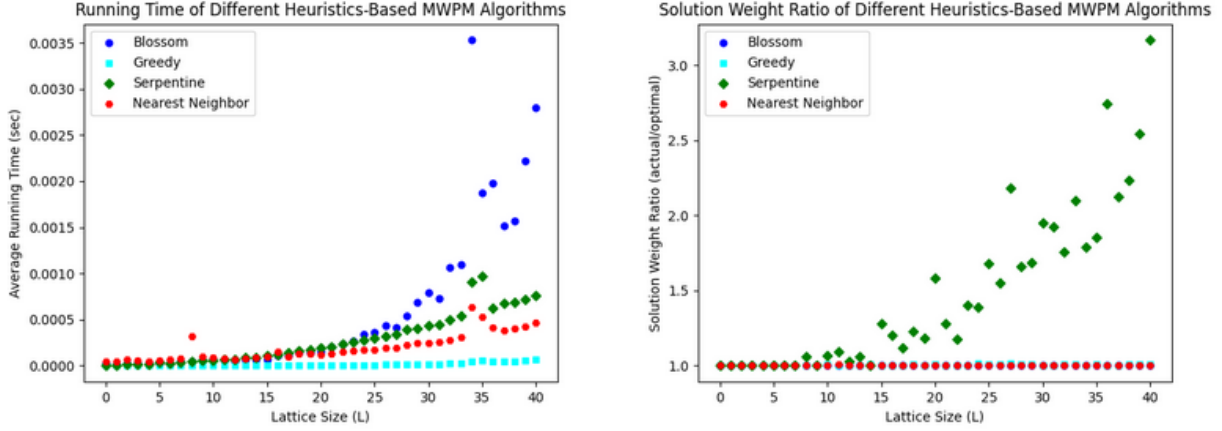


Figure 9: Average running time and error bound for $p = 0.01$ independent error model.

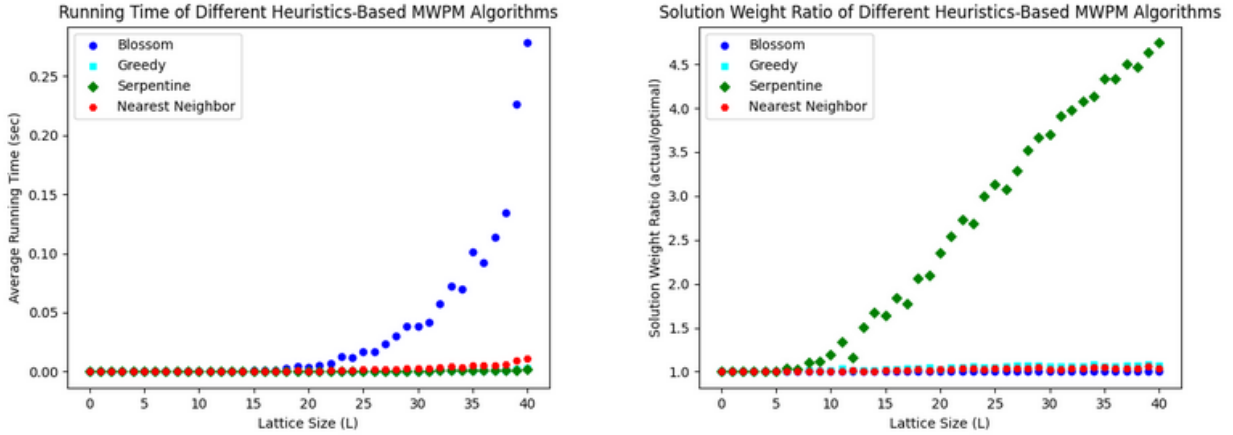


Figure 10: Average running time and error bound for $p = 0.05$ independent error model.

All three heuristics run faster than the pure blossom algorithm. In particular, the greedy heuristic works surprisingly well given its theoretical $O(N^2 \log N)$ running time. The error bounds are relatively small in both cases. However, the serpentine curve heuristic had too large an error bound to be realistic. Other space-filling curves, such as a Hilbert curve, may perform better.

The results of the analysis without the serpentine heuristic are shown below in Figure 11 and Figure 12. At both $p = 0.01$ and $p = 0.05$, the greedy heuristic slightly outperforms the nearest-neighbor heuristic in terms of speed. At $p = 0.01$, the greedy heuristic's error bound is also more volatile than the nearest-neighbor heuristic. However, at $p = 0.05$, the nearest-neighbor heuristic starts to clearly outperform the greedy heuristic in terms of error bound: when $L = 40$, the nearest-neighbor heuristic is around 5% costlier than the optimal solution, while the greedy heuristic is around 8% costlier. Realistically, this is a small margin and probably won't make a large difference in practice. Both heuristics are good candidates over the pure blossom algorithm.

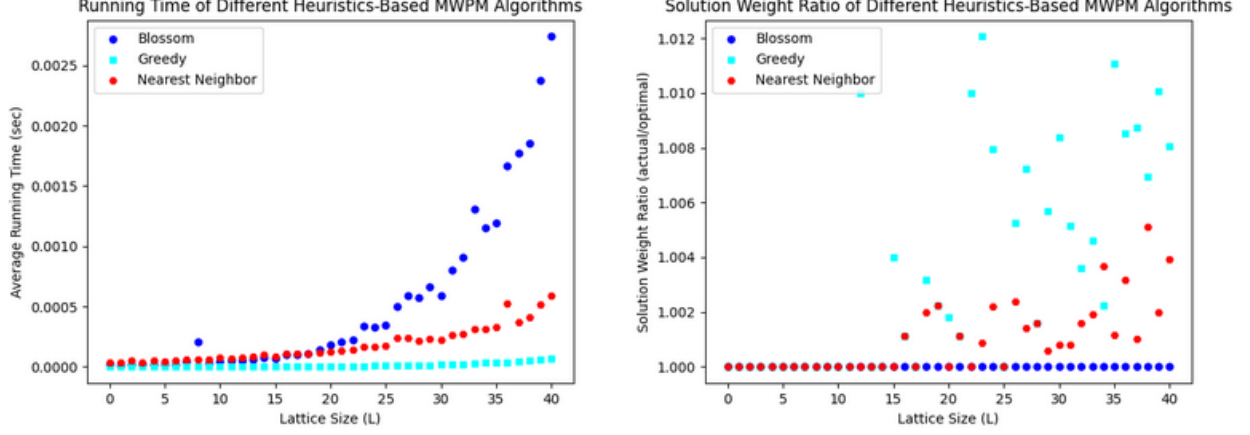


Figure 11: Average running time and error bound for $p = 0.01$ independent error model.

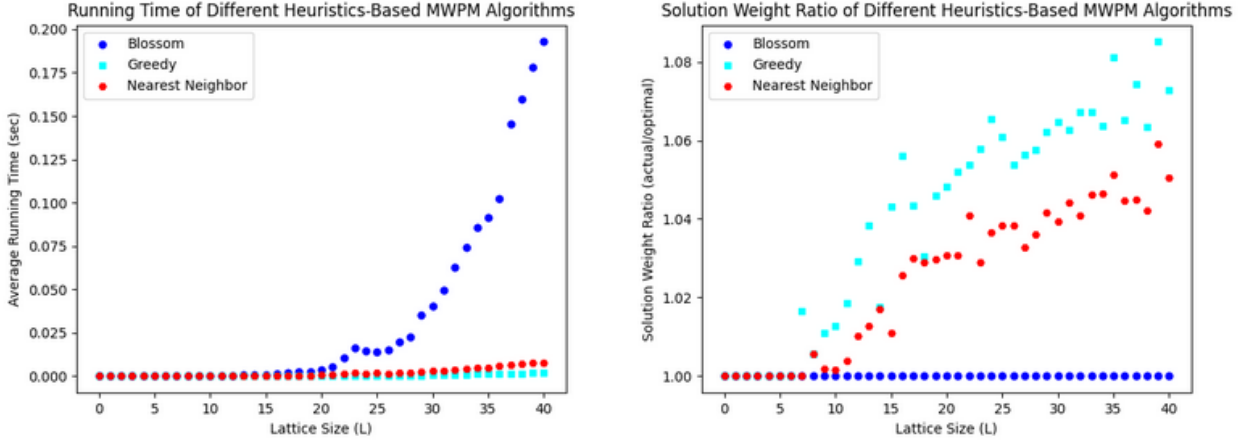


Figure 12: Average running time and error bound for $p = 0.05$ independent error model.

4.2 Correlated Errors Model

We can also consider a correlated error model. A basic simulation would be by raising the probability of a qubit error if its neighbor has an error. Keeping the base error rate p constant, a new “correlated error probability” p_{corr} can be introduced. While distributing errors on the lattice with probability p , another error can be applied to a qubit with probability p_{corr} if it neighbors an existing error. This will tend to lengthen the error chains present on the lattice.

Figure 13 shows the case of $p = 0.05$ with $p_{\text{corr}} = 0.10$ ($p = 0.01$ is similar to the uncorrelated error model). Surprisingly, not much has changed in the solution weight ratio: the greedy heuristic stays at a ratio of around 1.08, while the nearest-neighbor heuristic stays around 1.05. The greedy heuristic still outperforms the nearest-neighbor heuristic in terms of speed, but finds less optimal solutions.

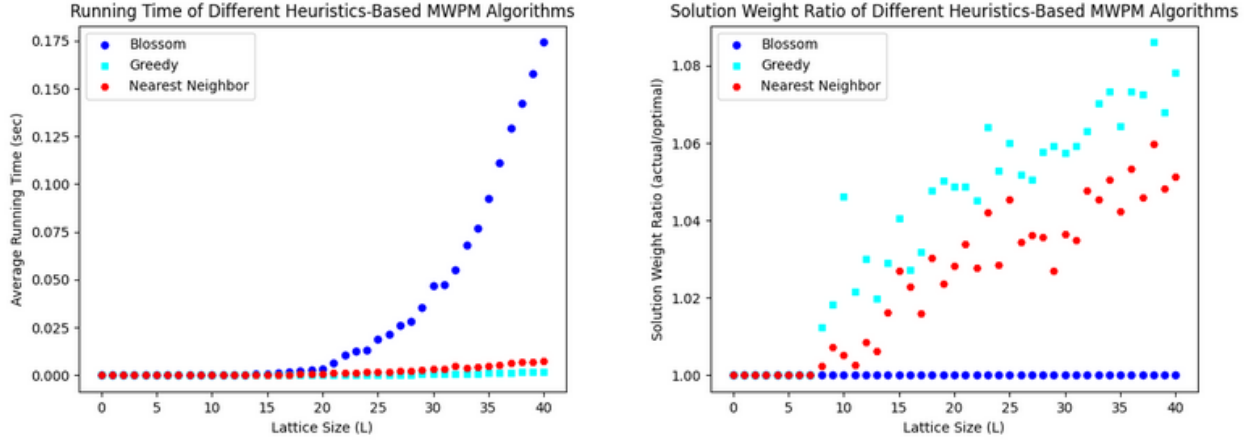


Figure 13: Average running time and error bound for $p = 0.05$, $p_{\text{corr}} = 0.10$ independent error model. The greedy heuristic is still faster than the nearest-neighbor heuristic, but the nearest-neighbor heuristic has a lower error bound.

5 Conclusion

Of the three heuristics analyzed (greedy, nearest-neighbor, serpentine), the greedy and nearest-neighbor heuristics performed quite well. Both of them led to substantial improvements in running time over the blossom algorithm, while staying within a cost that is within 8% and 5%, respectively, to the optimal (assuming $p = 0.05$ and $p_{\text{corr}} = 0.10$). For the relatively small lattices used in surface codes, these error bounds seem to be an acceptable tradeoff for the large increase in speed.

The simulations done here are crude. To give a more accurate account of heuristics performance, the code would need to be optimized and rigorous benchmarking beyond time-performance is required. Additionally, while the paper tries to analyze a variety of heuristics types (greedy, subgraph, geometric), there are several other heuristics that haven't been tested. It would be interesting to implement the Onethird or (t, k) heuristics, for example, and see if the complexity of their implementation is worth the potential speedup.

Several improvements to MWPM have been made without heuristics. Oscar Higgott's PyMatching v2 library, for example, returns an exact solution to the MWPM problem [12]. PyMatching v2 claims to give a staggering 400,000x speedup over NetworkX, the implementation for Edmond's blossom algorithm that was used in this paper. A future direction could be to test whether the use of heuristics can improve this speedup further.

References

- [1] A.Yu. Kitaev. "Fault-tolerant quantum computation by anyons". In: *Annals of Physics* 303.1 (Jan. 2003), pp. 2–30. ISSN: 0003-4916. DOI: 10.1016/S0003-4916(02)00018-0. URL: [http://dx.doi.org/10.1016/S0003-4916\(02\)00018-0](http://dx.doi.org/10.1016/S0003-4916(02)00018-0).
- [2] Nicolas Delfosse and Naomi H. Nickerson. "Almost-linear time decoding algorithm for topological codes". In: *Quantum* 5 (Dec. 2021), p. 595. ISSN: 2521-327X. DOI: 10.22331/q-2021-12-02-595. URL: <http://dx.doi.org/10.22331/q-2021-12-02-595>.
- [3] H. Bombín. *An Introduction to Topological Quantum Codes*. 2013. arXiv: 1311.0277 [quant-ph]. URL: <https://arxiv.org/abs/1311.0277>.

- [4] Austin G. Fowler et al. “Surface codes: Towards practical large-scale quantum computation”. In: *Physical Review A* 86.3 (2012). ISSN: 1094-1622. DOI: 10.1103/physreva.86.032324. URL: <http://dx.doi.org/10.1103/PhysRevA.86.032324>.
- [5] S. B. Bravyi and A. Yu. Kitaev. *Quantum codes on a lattice with boundary*. 1998. arXiv: quant-ph/9811052 [quant-ph]. URL: <https://arxiv.org/abs/quant-ph/9811052>.
- [6] J.P. Bonilla Ataides et al. “The XZZX surface code”. In: *Nature Communications* 12.2172 (2021). DOI: 10.1038/s41467-021-22274-1.
- [7] Rajeev Acharya et al. “Suppressing quantum errors by scaling a surface code logical qubit”. In: *Nature* 614.7949 (Feb. 2023), pp. 676–681. ISSN: 1476-4687. DOI: 10.1038/s41586-022-05434-1. URL: <http://dx.doi.org/10.1038/s41586-022-05434-1>.
- [8] Eric Dennis et al. “Topological quantum memory”. In: *Journal of Mathematical Physics* 43.9 (2002), pp. 4452–4505. ISSN: 1089-7658. DOI: 10.1063/1.1499754. URL: <http://dx.doi.org/10.1063/1.1499754>.
- [9] David K. Tuckett, Stephen D. Bartlett, and Steven T. Flammia. “Ultrahigh Error Threshold for Surface Codes with Biased Noise”. In: *Physical Review Letters* 120.5 (Jan. 2018). ISSN: 1079-7114. DOI: 10.1103/physrevlett.120.050505. URL: <http://dx.doi.org/10.1103/PhysRevLett.120.050505>.
- [10] Oscar Higgott. *PyMatching: A Python package for decoding quantum codes with minimum-weight perfect matching*. 2021. arXiv: 2105.13082 [quant-ph]. URL: <https://arxiv.org/abs/2105.13082>.
- [11] Harold N. Gabow. “An Efficient Implementation of Edmond’s Algorithm for Maximum Matching on Graphs”. In: *Journal of the Association for Computing Machinery* 23.2 (1976), pp. 221–234.
- [12] Oscar Higgott and Craig Gidney. “Sparse Blossom: correcting a million errors per core second with minimum-weight matching”. In: *Quantum* 9 (Jan. 2025), p. 1600. ISSN: 2521-327X. DOI: 10.22331/q-2025-01-20-1600. URL: <http://dx.doi.org/10.22331/q-2025-01-20-1600>.
- [13] Yue Wu, Namitha Liyanage, and Lin Zhong. *An interpretation of Union-Find Decoder on Weighted Graphs*. 2022. arXiv: 2211.03288 [quant-ph]. URL: <https://arxiv.org/abs/2211.03288>.
- [14] Giacomo Torlai and Roger G. Melko. “Neural Decoder for Topological Codes”. In: *Physical Review Letters* 119.3 (2017). ISSN: 1079-7114. DOI: 10.1103/physrevlett.119.030501. URL: <http://dx.doi.org/10.1103/PhysRevLett.119.030501>.
- [15] Edward M. Reingold and Robert E. Tarjan. “On a Greedy Heuristic for Complete Matching”. In: *SIAM Journal on Computing* 10.4 (1981), pp. 676–681.
- [16] M. D. Grigoriadis and B. Kalantari. “A New Class of Heuristic Algorithms for Weighted Perfect Matching”. In: *Journal of the Association for Computing Machinery* 35.4 (1988), pp. 769–776.
- [17] C. Imielińska and B. Kalantari. “A General Class of Heuristics for Minimum Weight Perfect Matching and Fast Special Cases with Doubly and Triply Logarithmic Errors”. In: *Algorithmica* 18 (1997), pp. 544–559.
- [18] John J. Bartholdi and Loren K. Platzman. “A fast heuristic based on spacefilling curves for minimum-weight matching in the plane”. In: *Information Processing Letters* 17.4 (1983), pp. 177–180. ISSN: 0020-0190. DOI: [https://doi.org/10.1016/0020-0190\(83\)90036-4](https://doi.org/10.1016/0020-0190(83)90036-4).
- [19] Masao Iri, Kazuo Murota, and Shouichi Matsui. “Linear-Time Approximation Algorithms for Finding the Minimum-Weight Perfect Matching on a Plane”. In: *Information Processing Letters* 12.4 (1981), pp. 206–209.
- [20] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. “Exploring Network Structure, Dynamics, and Function using NetworkX”. In: *Proceedings of the 7th Python in Science Conference*. Ed. by Gaël Varoquaux, Travis Vaught, and Jarrod Millman. Pasadena, CA USA, 2008, pp. 11–15.